

Plain-20 vs ResNet-20

用可稽核的正式 run 檢驗 residual shortcut 是否改善深度網路的最佳化

正式結果

8.51% test error

ResNet-20 · update 64,000 · seed 1

研究目的

研究問題不是「更深就一定更好」

在相同 depth、width、parameter count 與 training recipe 下，加入 residual shortcut 是否能降低最佳化困難？

degradation problem

深度增加時，plain network 可能出現更高 training error；這不是單純 overfitting，而是最佳化變難。

本研究的控制變因

同一 CIFAR-10 split、同一 20-layer 規格、同一 batch/LR schedule，只保留 shortcut 作為核心結構差異。

研究輸出

formal test error + training dynamics + reproducibility evidence

核心機制

Residual block 讓學習目標變成「修正量」

兩個模型的主分支相同；ResNet-20 額外把輸入 x 以無參數 shortcut 加回主分支。

Plain-20



Residual block



要點：shortcut 不增加 learnable parameters，因此 plain / residual 可做結構公平對照。

模型規格

Plain-20 與 ResNet-20 共享同一個 20-layer 骨架

CIFAR-10 stem + 三個 stage + global average pooling + 10-way classifier



Each block = 2 conv layers; stage transitions use stride 2

WEIGHTED DEPTH

20

CONVOLUTION + FC

19 + 1

SHORTCUTS

0 / 9

PARAMETERS

0.27M

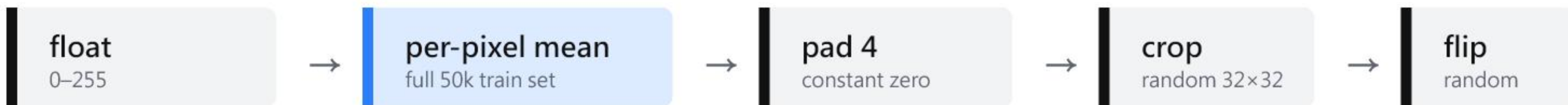
Plain-20 : 0 shortcuts ResNet-20 : 9 個 option A shortcuts · 無 trainable projection

資料流程

Preprocessing 固定了資料尺度與 augmentation 邊界

正式 run 沿用 frozen config：不做 channel-wise std normalization，test 只看原始 32×32 單一視角。

Training



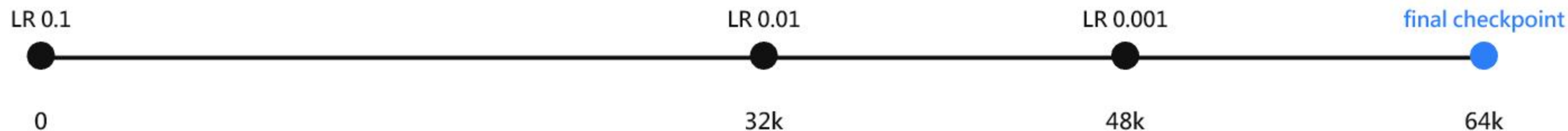
Test



訓練設定

訓練 recipe 被凍結在 64,000 次 optimizer update

核心設定：SGD、batch 128、momentum 0.9、weight decay 0.0001、seed 1、fp32。



test selection forbidden

所有報告數字固定取 update_64000_final ; evaluation logs 僅作觀測，不回頭挑 best test checkpoint。

FORMAL RUNS

2

TRAIN SAMPLES

8.192M

TEST SAMPLES

10,000

DEVICE

RTX 3070 Ti

可重現性

正式訓練前，證據鏈先通過 gates

preflight_v2 不是裝飾：它把 config、architecture、BN semantics、environment 與 data artifact 綁在一起。

TEST GATE

150 passed

pytest returncode 0 · BN compatibility PASS

HASH / CONFIG

config + mean artifact

[B6E9AA1...](#) / [6DAFA62...](#)

environment fingerprint

02AFBFC...

GIT 語境

formal source

e32662f...

freeze HEAD/tag

f48d29b...

重要區分：run manifest 記錄 source_dirty = true；freeze commit 是交付封存點，不等於 formal run 的 source snapshot。

結果表

正式結果：ResNet-20 在同一 recipe 下少 1.06 個百分點

所有數值取 final checkpoint at update 64,000 ; test set n = 10,000 ; used_for_selection = false 。

Model	Test error	Correct	Mean loss	Reporting basis
Plain-20 formal	9.57%	9043/10,000	0.3913	final @ 64k
ResNet-20 formal	8.51%	9149/10,000	0.3616	final @ 64k
ResNet-20 paper	8.75%	—	—	Table 6; form unknown

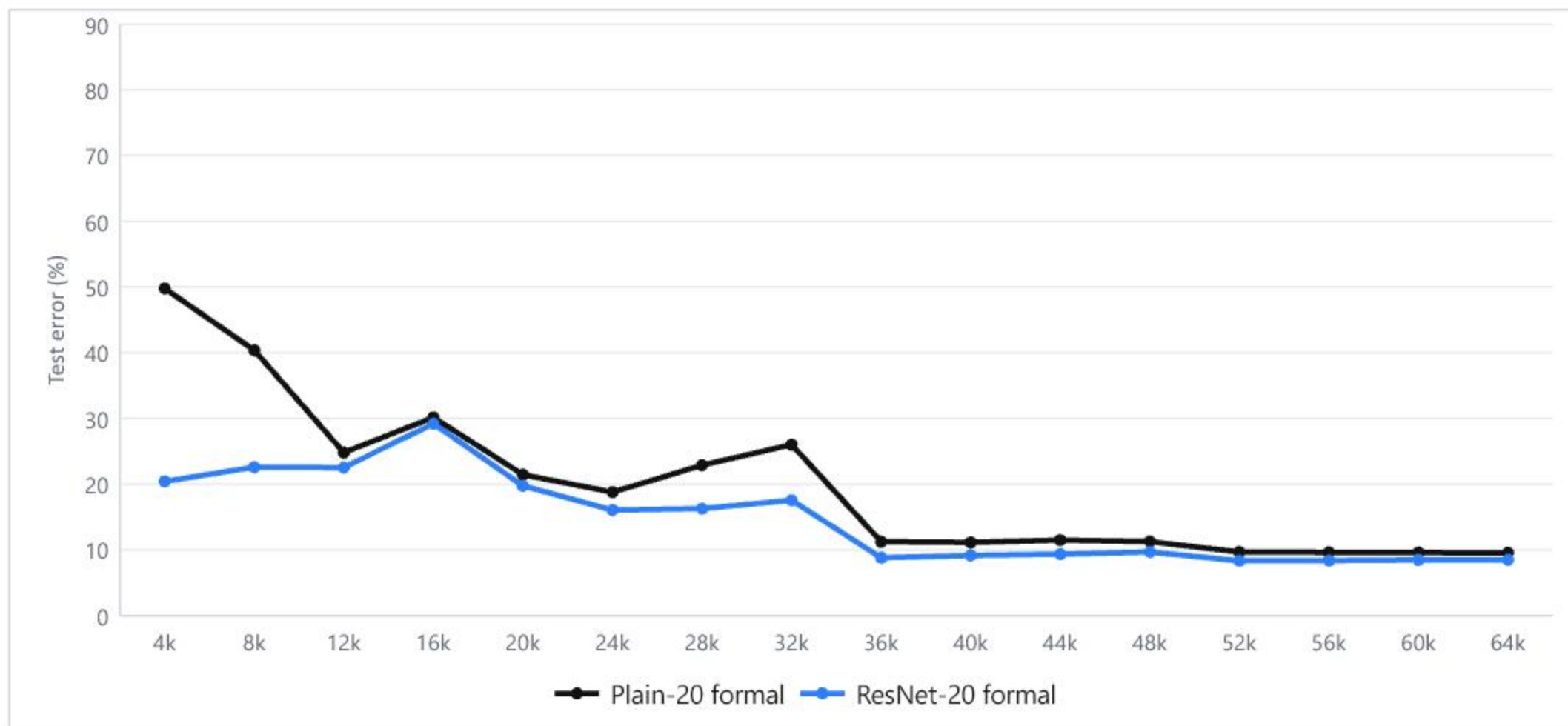
ResNet advantage: 1.06 pp lower error (11.1% relative reduction vs Plain-20 formal)

與論文 8.75% 的比較是透明的 numerical proximity，不宣稱 checkpoint / reporting form 完全一致。

訓練曲線

Residual run 從早期就呈現較低 test error

每 1,000 updates 評估一次；圖中以每 4,000 updates 的實際 evaluation point 標示，避免標籤擁擠。



at 1k

**54.04% vs
81.73%**

at 64k

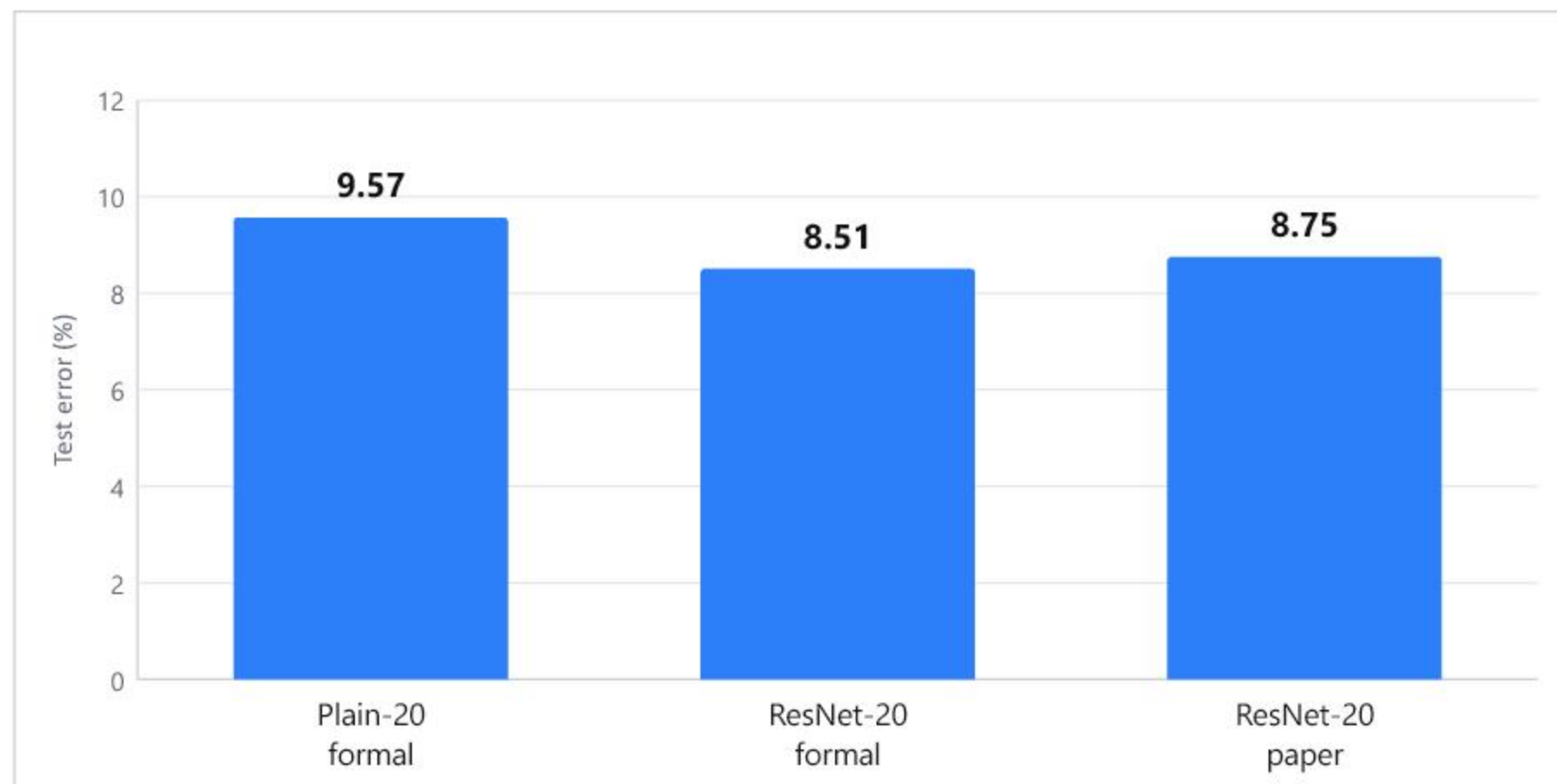
**8.51% vs
9.57%**

兩個 run 都在 32k / 48k
LR boundary
後下降；ResNet-20
全程多數觀測點保持較
低 test error。

誤差比較

結果比較：正式 ResNet-20 接近論文表列值

同一指標：CIFAR-10 test error (%)。Paper row 只提供單一表列值，沒有揭露 ResNet-20 的 best / last / mean 形式。



formal ResNet-20

8.51%

paper Table 6

8.75%

差距：-0.24 pp (formal 較低)

分析與限制

結果支持 residual shortcut 有效，但不能把差距解讀成完整複製

可支持的解讀

- 在本專案固定的同 recipe、同 seed、同 final-checkpoint 規則下，ResNet-20 test error 較低。
- 1.06 pp absolute improvement 與較低的 early test error，和「shortcut 幫助最佳化」一致。

不能過度宣稱

- 論文使用 two GPUs；正式 run 使用單張 RTX 3070 Ti。
- preprocessing order、BN details、Option A tensor indexing 等部分是 project assumptions。
- formal manifest 記錄 source_dirty = true；freeze commit 是後續交付封存點。

最重要的邊界

我們可以說：本專案的 formal ResNet-20 結果接近、且略低於論文 8.75%。
我們不能說：所有數值、checkpoint semantics、hardware execution 與作者原始 run 完全相同。

結論：在可稽核的 formal run 中， residual shortcut 帶來更低的 test error

9.57% → 8.51% · 1.06 pp

已完成

formal Plain-20 / ResNet-20 runs · final evaluations · training curves · preflight gates · manifests / hashes

下一步

把 source_dirty 對應的 formal implementation 另行整理成可重建 snapshot；再補做 multi-seed / hardware sensitivity，才能更強地討論穩健性。